$ javac NotificationLab.java && java NotificationLab

Java Interface

// □□□□□□□□□□□

contract
□譯期保證

dispatch
執行期選擇

injection
組裝端決策

```
$ cat learning-path.md
```

□日路線

01 / □□

interface □ implements

02 / □□

□面型別與真實物件

03 / □□

constructor injection、DIP、ISP

04 / □□

default、lambda、sealed

目標：能畫出依賴方向，也能用測試替身驗證高層流程。

PART 01

□找到耦合點

□□□□□□□□□□□□□□□□□□□□

第一版：能跑，但知道太多

```java
class NotificationService {
    void notify(String channel, String message) {
        if (channel.equals("email")) {
            new EmailClient().deliver(message);
        } else if (channel.equals("sms")) {
            new SmsClient().deliver(message);
        }
    }
}
```

COUPLING REPORT

- × 知道有哪些管道
- × 知道 SDK 如何建立
- × 知道每個 SDK 的方法名

新增 Push → 修改高層流程

把共同需求命名

不同技術細節

→

DOMAIN LANGUAGE

Notifier

`send(message)`

□面先回答：□叫端需要什麼行為?

宣告契約

```java
public interface Notifier {
    void send(String message);
}
```

public abstract
一般無本體方法隱含

no instance state
不能放每個物件的可變欄位

role name
描述能力，不綁供應商

`contract` ≠ 實作；它是□譯器可檢查的承諾。

`implements`：履行承諾

```java
final class EmailNotifier implements Notifier {
    @Override
    public void send(String message) {
        System.out.println("EMAIL > " + message);
    }
}

final class SmsNotifier implements Notifier {
    @Override
    public void send(String message) {
        System.out.println("SMS > " + message);
    }
}
```

[illegible] `public`；[illegible] package-private。

PART 02

同一型別，不同行為

□□□□□□□□□□□□□□

動態方法派送

```
1   Notifier notifier = new EmailNotifier();
2   notifier.send("Welcome");
3
4   notifier = new SmsNotifier();
5   notifier.send("Code: 8042");
```

COMPILE TIME

`Notifier` 有沒有 `send`？

RUN TIME

真實物件是哪個實作?

同一個呼叫語法 → 不同的方法本體

從具體型別移到契約

```
final class NotificationService {
    private final Notifier notifier;

    NotificationService(Notifier notifier) {
        this.notifier = notifier;
    }

    void notify(String message) {
        notifier.send(message);
    }
}
```

變化不只在型別：建立哪個實作的決策已移出服務。

建構子注入

```java
final class NotificationService {
    private final Notifier notifier;

    NotificationService(Notifier notifier) {
        this.notifier = notifier;
    }
}
```

`final`，物件建好後就是有效狀態。

組裝端

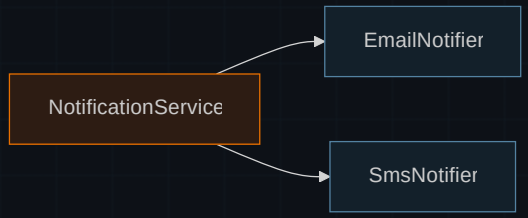
```java
public static void main(String[] args) {
    Notifier notifier =
        new EmailNotifier();

    var service =
        new NotificationService(notifier);
}
```

COMPOSITION ROOT

具體選擇集中在程式進入點。

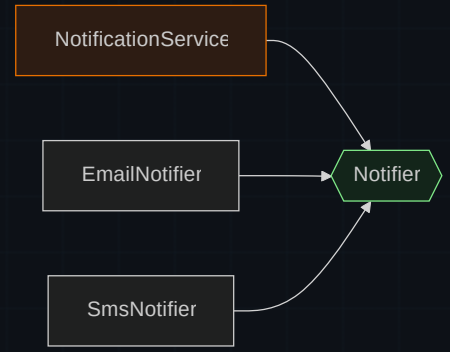
依賴方向


NotificationService

EmailNotifier

SmsNotifier

直接依賴：高層指向低層細節


NotificationService

EmailNotifier

SmsNotifier

Notifier

反轉後：兩側都指向契約

動手切換設計

拖曳我：新增管道時，哪個方塊必須改?

直接耦合

依賴契約

Email

SMS

Push

```
$ deps --graph
Service ⟶ Notifier ⟵ implementations
[illegible] → [illegible] Notifier[illegible]於組裝端接線
```

▶ dispatch()

```
// select channels, then dispatch
```

PART 03

原則不是口號

DIP□ISP □□□□□□□

□□□ interface□≠□□□ DIP」

過度宣稱

每個具體類別加一個同名□面，所以架構已解耦。

較準確

由高層政策需要定義抽象，低層細節反過來實作它。

問□不是「有沒有□鍵字」，而是□ 抽象由誰主導？原始碼箭頭指向哪裡？

CHECKPOINT_01

哪一句最準確描述 DIP?

A 只要宣告 interface，就自動符合 DIP

B 高層政策與低層細節都依賴由高層需求主導的抽象

C 每個 class 都必須建立同名 interface

ISP 別逼實作者說謊

```java
interface NotificationPlatform {
    void send(String message);
    void schedule(String message, Instant at);
    void revoke(String id);
}
```

SMS [redacted] `revoke` [redacted] `UnsupportedOperationException` 只是把設計問題延後到執行期。

`Notifier` `Schedulable` `Revocable`

組合：一個契約，多個委派

```java
final class CompositeNotifier implements Notifier {
    private final List<Notifier> delegates;

    CompositeNotifier(List<Notifier> delegates) {
        this.delegates = List.copyOf(delegates);
    }

    public void send(String message) {
        for (Notifier notifier : delegates) {
            notifier.send(message);
        }
    }
}
```

呼叫端仍只看到一個 `Notifier`。但要另外決定：某管失敗時，停止還是繼續？

□□□□

```java
final class RecordingNotifier
    implements Notifier {

    final List<String> messages =
        new ArrayList<>();

    public void send(String message) {
        messages.add(message);
    }
}
```

□□□□□□□

```java
@Test
void sendsMessage() {
    var fake = new RecordingNotifier();
    var service =
        new NotificationService(fake);

    service.notify("shipped");

    assertEquals(
        List.of("shipped"),
        fake.messages);
}
```

fast · deterministic · offline

Interface □□ abstract class□

問題	Interface	Abstract class
主要意圖	能力、邊界、替換	共同狀態與骨架
實例欄位	不可	可以
建構子	不可	可以
類別可擁有	多個□面	單一父類別
共用實作	□□□ `default`	完整支援

不要背「永遠優先□面」。共享不變量與建構流程時，抽象類別可能更直接。

□□ OOP 定位：LucasHsu.dev / Java / 多型與抽象類別

PART 04

介面如何演進

Java 8、9、17 的變化

default ▌NOGLYPH▐ static Java 8+

```java
interface Notifier {
    void send(String message);

    default void sendUrgent(String message) {
        send("[URGENT] " + message);
    }

    static Notifier silent() {
        return message -> { };
    }
}
```

default

▒▒▒▒▒▒▒▒▒▒▒▒ API 演進

static

屬於□面本身；不參與動態派送

`private` interface method Java 9+

```java
interface Notifier {
    default void sendUrgent(String message) {
        send("[URGENT] " + normalize(message));
    }

    default void sendNormal(String message) {
        send(normalize(message));
    }

    private String normalize(String message) {
        return message.strip();
    }

    void send(String message);
}
```

`private` 方法只供介面內部重用；實作類別不可見。

Functional Interface + lambda Java 8+

```java
@FunctionalInterface
interface Notifier {
    void send(String message);
}

Notifier console =
    message -> System.out.println(message);
```

一個抽象方法

default / static / private 不計入

@FunctionalInterface

讓編譯器保護設計意圖

Sealed interface Java 17 正式版

```java
sealed interface DeliveryResult
    permits Delivered, Rejected, Retrying {}

record Delivered(String id) implements DeliveryResult {}
record Rejected(String reason) implements DeliveryResult {}
record Retrying(int attempt) implements DeliveryResult {}
```

適合

封閉的領域變體集合

不適合

[illegible] plugin API

Java 15[illegible]16 [illegible]JEP 409[illegible] Java 17 定版。

Monaco 練習：補上契約

[illegible] Java[illegible]

```java
interface Notifier {
    // TODO: 宣告 send(String message)
}

final class PushNotifier implements Notifier {
    // TODO: 實作[illegible]印出 "PUSH > " + message
}

final class NotificationService {
    // TODO: 以建[illegible]入 Notifier
}
```

設計檢查清單

01 □□□□□□□□□□□□

02 □□□□□□□□□□□□□□□□□

03 □□□□□□□□□□□□□

04 □□□□□□□□□□□□□□□□

05 □□□□□□□□ fake□

版本速查

能力	版本	用途
`default` / `static` methods	Java 8	預設行為、□面工廠
Functional interface / lambda	Java 8	單一行為的精簡實作
`private` interface methods	Java 9	□面內部重用
Sealed interfaces	Java 17	限制直接實作集合

規格來源: JLS §9 · JLS §9.8 · JEP 409

□日重點

契約

描述呼叫端需要的能力

派送

同一呼叫，依真實物件執行

依賴

選擇移到組裝端；箭頭指向抽象

測試

□□ fake□隔離外部服務

`Interface` 不會消除變更；它讓變更邊界更清楚。

Lab complete.

□□□□□ FailingNotifier □□□□□□

```
$ java NotificationLab --reflect
contract → dispatch → injection → test
```